

Programación Dinámica Avanzada

~~Gabriel Carmona~~

Blaz Korecic



Resumen de la clase

Repaso

- Estados y espacio de estados

Optimizando las transiciones con estructuras

- Pillars (CF 474E)

Parte 1: Exponenciación binaria y recurrencias lineales

- Exponenciación binaria
- Recurrencias lineales con matrices

Parte 2: DP con estados especiales

- Bitmask DP
- Digit DP



Subproblemas y estados

La programación dinámica descompone un problema en **subproblemas** que se solapan y reutiliza sus resultados. Podemos formalizarlo con tres ingredientes:

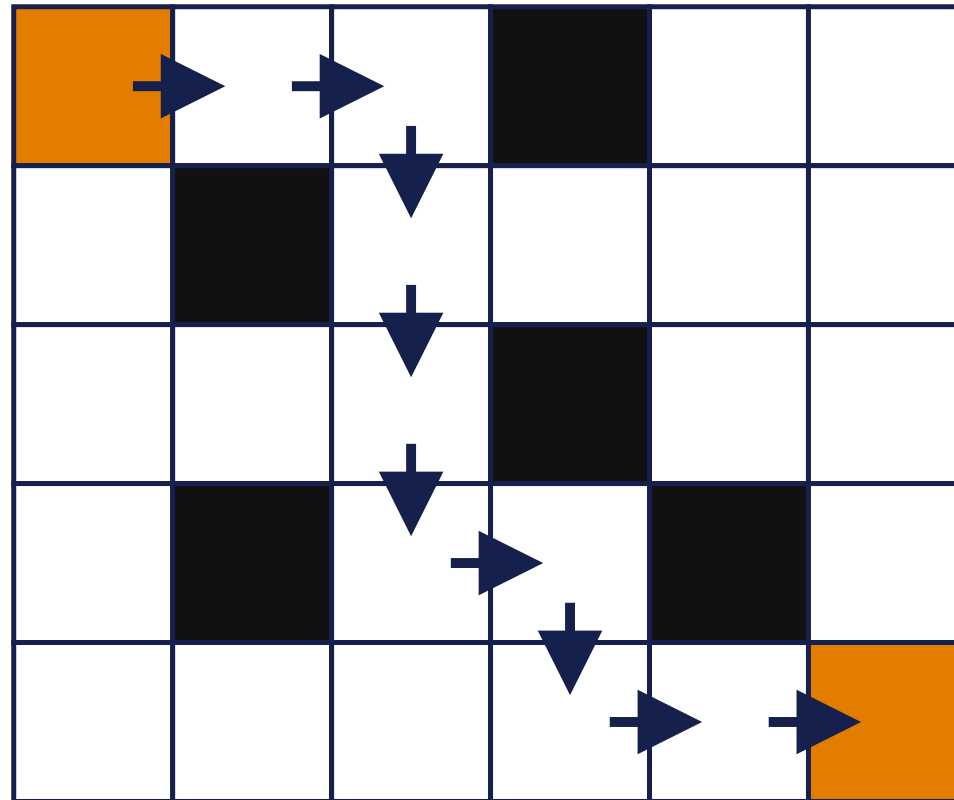
- un conjunto de **estados** S ,
- una **función de valor** $f : S \rightarrow V$ que queremos calcular,
- **transiciones** que expresan $f(s)$ en función de estados "más simples".

Resolver el problema es evaluar f en un estado objetivo, calculando cada estado **una sola vez**.



Hablemos de estados

Tenemos una grilla de $n \times m$ con algunas casillas **bloqueadas**. Partiendo en $(0, 0)$ y moviéndonos solo a la **derecha** o hacia **abajo** entre casillas libres, ¿cuántos **caminos válidos** llegan hasta $(n - 1, m - 1)$?





Un **estado** representa de forma única una configuración del problema mediante un conjunto de variables.

En este problema basta la **posición actual**: la tupla (i, j) de fila y columna. Cada casilla del dibujo es un estado, y las flechas muestran las **transiciones** de un camino posible.



La DP de la grilla

Sea $dp[i][j]$ el número de caminos desde $(0, 0)$ hasta (i, j) moviéndonos solo a la derecha o hacia abajo:

$$dp[i][j] = \begin{cases} 0 & \text{si } (i, j) \text{ está bloqueada} \\ 1 & \text{si } (i, j) = (0, 0) \\ dp[i-1][j] + dp[i][j-1] & \text{en otro caso} \end{cases}$$

Los términos fuera de la grilla valen 0. La cantidad de caminos válidos es $dp[n-1][m-1]$.



Otro ejemplo: CSES 1635

Tenemos n monedas de valores $c_1, \dots, c_n$, con repetición ilimitada, y un objetivo x .

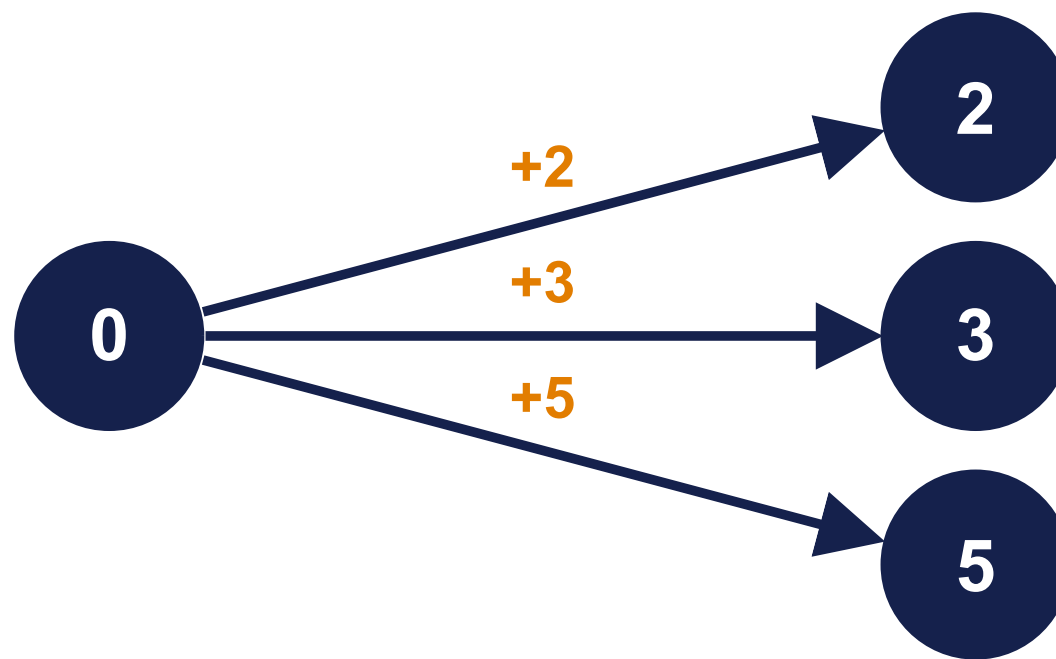
¿De cuántas **formas ordenadas** podemos sumar exactamente x ? Dos secuencias con las mismas monedas en distinto orden cuentan por separado.



Transiciones

Aquí el estado no es tan directo: hay que deducir que basta con la **suma acumulada** $s \in \{0, 1, \dots, x\}$.

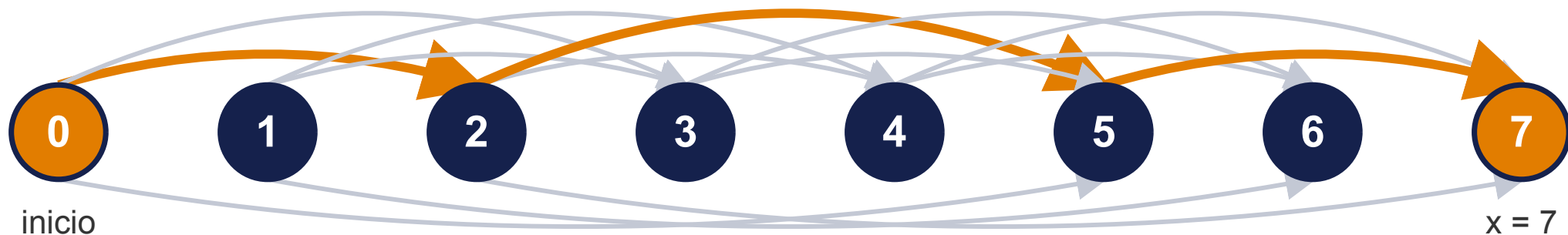
Con monedas $\{2, 3, 5\}$, desde $s = 0$ elegir una moneda c nos lleva a $s + c$: eso es una **transición**.





Espacio de estados

El espacio de estados es un **grafo dirigido**: un nodo por cada suma y una arista $s \rightarrow s + c$ por cada moneda.



Contar las formas de sumar x es contar **caminos** desde 0 hasta x . ¡Eso ya sabemos hacerlo!



La DP de las monedas

Sea $dp[s]$ el número de formas de sumar s :

$$dp[s] = \sum_{c: c \leq s} dp[s - c], \quad dp[0] = 1$$

La respuesta es $dp[x]$.

Optimizando las transiciones con estructuras



Enunciado (Pillars, CF 474E):

Hay n pilares en fila; el pilar i tiene altura h_i . Dado d , podemos saltar del pilar i al pilar j solo si $i < j$ y $|h_i - h_j| \geq d$.

Encuentra la secuencia de saltos de **mayor largo** posible (y reconstrúyela).

Link: <https://codeforces.com/problemset/problem/474/E>



Solución subóptima

$dp[i]$ = largo máximo de una secuencia que termina en el pilar i .

$$dp[i] = 1 + \max_{j < i, |h_i - h_j| \geq d} dp[j]$$

Corre en $O(n^2)$. ¿Podemos hacerlo mejor?



Recorramos los pilares de izquierda a derecha; así la restricción $j < i$ es trivial (solo consideramos lo ya visto).

La condición $|h_i - h_j| \geq d$ se parte en dos rangos de alturas:

- $h_j \leq h_i - d$
- $h_j \geq h_i + d$

Calcular $dp[i]$ es tomar el **máximo de dp sobre esos dos rangos de valores.**



Solución optimizada

Comprimimos las alturas y mantenemos un **Segment Tree** indexado por altura que guarda el máximo dp visto hasta el momento.

- Query: máximo en $(-\infty, h_i - d]$ y en $[h_i + d, +\infty)$
- Update: colocamos $dp[i]$ en la posición de h_i

Guardando además de qué posición viene el máximo, reconstruimos la secuencia.

Complejidad final: $O(n \log n)$

Parte 1

Exponenciación binaria y recurrencias lineales



Queremos calcular a^n de forma eficiente.

- *Naive*: multiplicar a por sí mismo n veces $\rightarrow O(n)$
- ¿Y si n es gigante (hasta 10^{18})?



Idea: cuadrados sucesivos

Observación clave:

$$a^n = \begin{cases} (a^{n/2})^2 & \text{si } n \text{ es par} \\ a \cdot a^{n-1} & \text{si } n \text{ es impar} \end{cases}$$

Cada paso reduce el exponente a la mitad $\rightarrow$ **$O(\log n)$** multiplicaciones.



Escribimos n en binario:

$$a^{13} = a^{1101_2} = a^8 \cdot a^4 \cdot a^1$$

Recorremos los bits de n manteniendo a^{2^k} (elevándolo al cuadrado en cada paso) y multiplicamos al resultado cuando el bit está encendido.



Implementación

```
ll binpow(ll a, ll n) {  
    ll res = 1;  
    while (n > 0) {  
        if (n & 1) res *= a;  
        a *= a;  
        n >>= 1;  
    }  
    return res;  
}
```

Complejidad: $O(\log n)$



Funciona con cualquier operación asociativa

Lo único que usamos es la **asociatividad** del producto. Sirve para:

- Enteros módulo M
- **Multiplicación de matrices** $\leftarrow$ lo que usaremos ahora
- Cualquier monoide

Esto conecta directamente con las **recurrencias lineales**.



Una **recurrencia lineal** es una recurrencia donde el k -ésimo término es una combinación lineal de términos anteriores.

Ejemplos:

- Fibonacci: $F(n) = F(n - 1) + F(n - 2)$

Esto significa que podemos escribirlas utilizando **matrices**!



Fibonacci en forma matricial

$$\begin{pmatrix} F(n) \\ F(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F(n-1) \\ F(n-2) \end{pmatrix}$$



Queremos calcular $F(n)$, el n -ésimo número de Fibonacci.

- Recurrencia sin memoizar: $O(2^n)$
- DP normal: $O(n)$
- **Exponenciación binaria: $O(\log n)$**



Si tenemos el vector con los valores $[F(n - 1), F(n - 2)]$, al multiplicarlo por la matriz M de recién shifteamos los valores en uno:

$$M \cdot \begin{pmatrix} F(n - 1) \\ F(n - 2) \end{pmatrix} = \begin{pmatrix} F(n) \\ F(n - 1) \end{pmatrix}$$

Intuitivamente, multiplicar por M^2 los shiftea en 2 y así sucesivamente. Se tiene que

$$M^n \cdot \begin{pmatrix} F(1) \\ F(0) \end{pmatrix} = \begin{pmatrix} F(n + 1) \\ F(n) \end{pmatrix}$$



Calcular la k -ésima potencia de una matriz de $m \times m$ tiene complejidad $O(m^3 \log k)$:

- Multiplicar matrices tiene complejidad $O(m^3)$
- En total se hacen $O(\log k)$ multiplicaciones

En conclusión, como la matriz es de 2×2 , hacemos aproximadamente $O(8 \log n)$ operaciones.



Problema de ejemplo

Un boss tiene h de vida. Tienes n hechizos; el hechizo i quita d_i de vida.

¿De cuántas formas (**secuencias ordenadas** de hechizos) se puede llevar la vida de h exactamente a 0? (módulo M)

Restricciones: $1 \leq d_i \leq 100, h \leq 10^{18}, M = 998244353$.



Sea $W(v)$ = número de formas de llevar la vida de v a 0, con $W(0) = 1$. Sumando sobre el último hechizo usado:

$$W(v) = \sum_{\delta=1}^D k_{\delta} W(v - \delta)$$

donde k_{δ} = número de hechizos de daño δ (**fijo**) y D = daño máximo.

Es una **recurrencia lineal de coeficientes constantes** de orden D . Como $h \leq 10^{18} \rightarrow$ **exponenciación de matrices**.



Armando la matriz

Cualquier recurrencia lineal $a_n = k_1 a_{n-1} + k_2 a_{n-2} + k_3 a_{n-3}$ se vuelve un producto **matriz-vector**, con el vector de estado = los últimos D valores:

$$\begin{bmatrix} a_n \\ a_{n-1} \\ a_{n-2} \end{bmatrix} = \begin{bmatrix} k_1 & k_2 & k_3 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \cdot \begin{bmatrix} a_{n-1} \\ a_{n-2} \\ a_{n-3} \end{bmatrix}$$

 coeficientes
→ calculan a_n

 identidad: copia
(shift) los viejos

La fila de arriba **calcula** a_n ; las de abajo **desplazan** los valores. Exponenciando M saltamos muchos pasos: $\mathbf{v}_n = M^k \mathbf{v}_{\text{base}}$.

Parte 2

DP con estados especiales



Repaso: operaciones de bits

Operación	Código
Prender bit i	<code>mask (1 << i)</code>
Apagar bit i	<code>mask & ~(1 << i)</code>
Togglear bit i	<code>mask ^ (1 << i)</code>
Chequear bit i	<code>(mask >> i) & 1</code>
Bits prendidos	<code>__builtin_popcount(mask)</code>
Bit menos significativo	<code>mask & (-mask)</code>
Iterar todas las máscaras de n bits	<code>for (int mask = 0; mask < (1 << n); mask++)</code>
Iterar submáscaras de <code>mask</code>	<code>for (int sub = mask; sub > 0; sub = (sub-1) & mask)</code>



Cuando las constraints son pequeñas ($n \leq 20$), podemos usar una cantidad **exponencial** de estados: uno por cada subconjunto.

Ejemplo: TSP (Traveling Salesman Problem)



Tenemos n ciudades ($n \leq 20$) conectadas por aristas con peso. Queremos encontrar un **camino hamiltoniano de costo mínimo**.

Hay $O(n!)$ caminos posibles, exploración exhaustiva imposible para n grande.



El estado codifica qué ciudades ya visitamos y en cuál terminamos:

$dp(u, S)$ = mínima distancia para visitar exactamente las ciudades de S , terminando en u

Base: $dp(u, \{u\}) = 0$ para todo u

Transición:

$$dp(u, S) = \min_{v \in S \cap N(u)} \{dp(v, S \setminus \{u\}) + w(u, v)\}$$

Respuesta: $\min_{u \in V} \{dp(u, V)\}$

Complejidad: $O(2^n \cdot n^2)$



Enunciado:

Hay n personas ($n \leq 20$) con pesos $w_1, \dots, w_n$ y un ascensor con límite de peso x .
¿Cuál es el **mínimo número de viajes** para subir a todos?

Link: <https://cses.fi/problemset/task/1690>



Guardemos solo lo esencial:

$dp[mask] = \text{mín. de viajes para subir a las personas de } mask$

Transición: elegimos qué subconjunto sube en **un** viaje. Si $sub \subseteq mask$ cabe en el ascensor ($\sum_{i \in sub} w_i \leq x$):

$$dp[mask] = \min_{sub \text{ válido}} 1 + dp[mask \setminus sub]$$

Complejidad: $O(3^n)$, hay 3^n pares $(mask, sub)$ en total. Correcto, pero lento.



En vez de elegir viajes completos, construyamos la solución **persona por persona**: eso baja a $O(n \cdot 2^n)$.

Pero al agregar a una persona necesito saber **cuánto peso lleva el viaje que estoy llenando**. Guardemos ese dato en el estado:

$$dp[mask] = (\text{viajes}, \text{peso usado en el último viaje})$$

minimizado **lexicográficamente**: menos viajes primero y, a igualdad, el último viaje lo más vacío posible (deja más espacio).



$dp[mask] = (\text{viajes}, \text{peso del último viaje})$, mínimo lexicográfico.

Para cada $p \in mask$ tomamos $(r, c) = dp[mask \setminus \{p\}]$ e insertamos a p :

$$\text{agregar}((r, c), p) = \begin{cases} (r, c + w_p) & \text{si } c + w_p \leq x \\ (r + 1, w_p) & \text{si } c + w_p > x \end{cases}$$

$$dp[mask] = \min_{p \in mask} \text{agregar}(dp[mask \setminus \{p\}], p)$$

Base: $dp[\emptyset] = (1, 0)$. **Respuesta:** primera componente de $dp[\{0, \dots, n - 1\}]$.



Implementación

```
pair<int,int> dp[1 << 20];  
dp[0] = {1, 0}; // 1 viaje vacío  
for (int mask = 1; mask < (1 << n); mask++) {  
    dp[mask] = {n + 1, 0}; // peor caso  
    for (int p = 0; p < n; p++) if (mask & (1 << p)) {  
        auto [rides, w] = dp[mask ^ (1 << p)];  
        if (w + weight[p] <= x) w += weight[p]; // cabe  
        else rides++, w = weight[p]; // nuevo viaje  
        dp[mask] = min(dp[mask], {rides, w});  
    }  
}  
// respuesta: dp[(1 << n) - 1].first
```



Queremos contar números en un rango $[L, R]$ que cumplen una propiedad de sus **dígitos**, con L, R hasta 10^{18} .

No podemos iterar número por número. La idea es construir el número **dígito a dígito**.



Problema motivador

Dado L y R , cuenta cuántos enteros en $[L, R]$ tienen **suma de dígitos prima**.

Restricciones: $0 \leq L \leq R \leq 10^{18}$.



Reducción del rango

Primero reducimos el rango a un solo techo. Sea:

$$f(N) = \#\{ x \in [0, N] : x \text{ cumple la propiedad } \}$$

Entonces la respuesta en $[L, R]$ es $f(R) - f(L - 1)$.



Construimos el número de izquierda a derecha. Para nuestro problema, el estado es:

- **pos**: posición del dígito actual
- **tight**: ¿el prefijo va pegado al límite N o ya quedó estrictamente menor?
- **sum**: suma de dígitos acumulada

Para otros problemas, `sum` se reemplaza por el dato relevante: resto módulo k , dígito previo, etc.



En cada posición elegimos el dígito d :

- Si **tight**: d puede ir de 0 hasta $N[pos]$.
 - Si $d = N[pos]$ seguimos **tight**.
 - Si $d < N[pos]$ nos **soltamos** (el resto de dígitos quedan libres).
- Si **no tight**: d va libre de 0 a 9.

Para nuestro problema: la suma pasa a $sum + d$.



Implementación

```
string dig = to_string(N);
int rec(int pos, bool tight, int sum) {
    if (pos == (int)dig.size()) return is_prime(sum);
    if (memo[pos][tight][sum] != -1)
        return memo[pos][tight][sum];
    int lim = tight ? dig[pos] - '0' : 9, total = 0;
    for (int d = 0; d <= lim; d++)
        total += rec(pos + 1, tight && d == lim, sum + d);
    return memo[pos][tight][sum] = total;
}
```



Aplicación: Counting Numbers

Enunciado:

Cuenta cuántos enteros en $[a, b]$ **no tienen dos dígitos adyacentes iguales**.

Restricciones: $0 \leq a \leq b \leq 10^{18}$

Link: <https://cses.fi/problemset/task/2220>

En vez de `sum` guardamos el **dígito previo**. A diferencia del problema anterior, los ceros a la izquierda sí importan: sin cuidado, "007" compararía dos ceros como dígitos adyacentes iguales. Para esto agregamos un flag `started` que indica si ya pusimos algún dígito distinto de 0.



Temas de DP avanzada que quedaron fuera de esta clase:

- **SOS DP** (Sum over Subsets): suma/mínimo sobre todos los subconjuntos de una máscara en $O(n \cdot 2^n)$
- **Convex Hull Trick**: optimiza DPs con transiciones lineales de $O(n^2)$ a $O(n \log n)$
- **Li Chao Tree**: alternativa a CHT que soporta consultas y líneas en cualquier orden